

End-to-End TEAVS–ADCRS Integration Verification

Purpose

The purpose of this test was to verify how far a valid TEAVS payload could travel through the PHOENIX integration path and to identify any remaining dependencies preventing full core-model processing.

The endpoint tested was:

POST /api/ingestion/core

The intended flow was:

TEAVS → API Gateway → RabbitMQ → data-ingestion-service →
coreModelIntegration() → trained model inference

Test Payload

A valid TEAVS payload was submitted using a valid ingestion_service JWT.

The test payload included the required fields defined by the current CoreModelIntegrationPayload interface, including:

- url
- text
- timestamp
- hazard_type
- hazard_severity
- hazard_timestamp
- hazard_location
- hazard_status
- alert_level
- source

A unique marker was included in the request:

TEAVS E2E TEST 002

This allowed the request to be traced through the downstream service logs.

API Gateway Result

The API Gateway accepted the request and returned:

HTTP 202 Accepted

Response:

Core model integration data sent successfully

This confirmed that the request successfully passed:

- JWT authentication
- RBAC
- rate limiting
- payload validation

and was accepted for downstream processing.

RabbitMQ and Consumer Verification

The data-ingestion-service logs confirmed that the payload was received from:

core_model_integration_queue

The logs included the unique marker:

TEAVS E2E TEST 002

The payload was then passed into the coreModelIntegration() processing function.

This confirmed that the integration path was successfully operating through:

API Gateway → RabbitMQ → data-ingestion-service

Database Issues Identified During Testing

During the initial end-to-end test, several local database schema dependencies were missing.

The following tables were not initially available:

- data_ingestion_streaming_log
- integration_log
- stored_file

The relevant Sequelize models already existed in the codebase, but the local project did not contain a migration directory or automatic sequelize.sync() process.

The missing tables were created in the local development database using the existing Sequelize model definitions.

After this, the data-ingestion-service was able to start normally and continue processing the core integration payload.

Final Processing Result

After the required database tables were available, the request successfully reached the core model integration service.

The service then attempted to load the configured model file:

final_core_xgb_xgboost_trey_xgb_core_v2_epoch_100.joblib

The service returned the following downstream error:

Training model not found

A repository search confirmed that the filename is referenced in:

- libs/common/src/helper/core_model_integration.py

- data-ingestion-service/src/services/ingestion.service.ts

However, no .joblib, .pkl, or .pickle model file was present in the local repository.

Integration Status

The end-to-end integration status at the conclusion of testing was:

Integration Stage	Result
TEAVS payload submitted	PASS
API Gateway acceptance	PASS
JWT authentication	PASS
RBAC enforcement	PASS
Input validation	PASS
RabbitMQ publishing	PASS
RabbitMQ consumer reception	PASS
data-ingestion-service processing	PASS
coreModelIntegration() execution	PASS
Model asset lookup	FAIL
Core-model inference	BLOCKED

Remaining Blocker

The remaining blocker is the absence of the trained model artifact:

`final_core_xgb_xgboost_trey_xgb_core_v2_epoch_100.joblib`

The integration code expects this model to be available through the stored_file table before inference can run.

No copy of the required model file was present in the repository or local development environment.

Because of this, full inference could not be verified.

Assessment

The testing confirmed that the TEAVS–ADCRS integration path is connected successfully up to the model-loading stage.

The request successfully travelled through:

TEAVS → API Gateway → RabbitMQ → data-ingestion-service →
coreModelIntegration()

The remaining issue is not related to API security, RBAC, request validation, RabbitMQ connectivity, or message routing.

The outstanding dependency is the availability of the trained model artifact required by the core-model integration service.

Evidence

EVID-E2E-01 – Valid TEAVS request accepted

```
(base) abdulrehman@Abduls-Air backend % export INGESTION_TOKEN=$(curl -s -X POST http://localhost:3001/api/users/auth/login \
-H "Content-Type: application/json" \
-d '{
  "username": "rbac_test_ingestion",
  "password": "PhoenixTemp2026iv"
}' | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
(base) abdulrehman@Abduls-Air backend % curl -i -X POST http://localhost:3001/api/ingestion/core \
-H "Authorization: Bearer $INGESTION_TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "url": "https://example.com/e2e-evidence",
  "text": "TEAVS E2E EVIDENCE TEST",
  "timestamp": "2026-09-27T09:50:00+10:00",
  "hazard_type": "phishing",
  "hazard_severity": 4,
  "hazard_timestamp": "2026-09-27T09:49:00+10:00",
  "hazard_location": "VIC",
  "hazard_status": "active",
  "alert_level": "critical",
  "source": "TEAVS"
}'
HTTP/1.1 202 Accepted
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 72
ETag: W/"48-RWbroSn8m+IA+xa+VR31/N5CFLM"
Date: Sat, 26 Sep 2026 23:37:31 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":202,"message":"Core model integration data sent successfully"}
(base) abdulrehman@Abduls-Air backend %
```

EVID-E2E-02 – RabbitMQ consumer receives TEAVS payload

```
(base) abdulrehman@Abduls-Air backend % docker compose logs --tail=120 data-ingestion-service | grep -i -E 'core_model_integration_queue|TEAVS E2E EVIDENCE TEST'
ingestion-service | Received message from core_model_integration_queue: {"url":"https://example.com/e2e-test-002","text":"TEAVS E2E TEST 002","timestamp":"2026-09-27T09:40:00+10:00","hazard_type":"phishing","hazard_severity":
4,"hazard_timestamp":"2026-09-27T09:39:00+10:00","hazard_location":"VIC","hazard_status":"active","alert_level":"critical","source":"TEAVS"}
ingestion-service | Received message from core_model_integration_queue: {"url":"https://example.com/e2e-evidence","text":"TEAVS E2E EVIDENCE TEST","timestamp":"2026-09-27T09:50:00+10:00","hazard_type":"phishing","hazard_sever
ity":4,"hazard_timestamp":"2026-09-27T09:49:00+10:00","hazard_location":"VIC","hazard_status":"active","alert_level":"critical","source":"TEAVS"}
ingestion-service | text: 'TEAVS E2E EVIDENCE TEST',
(base) abdulrehman@Abduls-Air backend %
```

EVID-E2E-03 – Core integration processing reached

```
(base) abdulrehman@Abduls-Air backend % docker compose logs --tail=120 data-ingestion-service | grep -i -A 15 -B 3 'Received core model integration data'
ingestion-service | RabbitMQ connected
ingestion-service | Data ingestion service gRPC running on port 50053
ingestion-service | Received message from core_model_integration_queue: {"url":"https://example.com/e2e-test-002","text":"TEAVS E2E TEST 002","timestamp":"2026-09-27T09:40:00+10:00","hazard_type":"phishing","hazard_severity":
4,"hazard_timestamp":"2026-09-27T09:39:00+10:00","hazard_location":"VIC","hazard_status":"active","alert_level":"critical","source":"TEAVS"}
ingestion-service | Received core model integration data: {
ingestion-service |   url: 'https://example.com/e2e-test-002',
ingestion-service |   text: 'TEAVS E2E TEST 002',
ingestion-service |   timestamp: '2026-09-27T09:40:00+10:00',
ingestion-service |   hazard_type: 'phishing',
ingestion-service |   hazard_severity: 4,
ingestion-service |   hazard_timestamp: '2026-09-27T09:39:00+10:00',
ingestion-service |   hazard_location: 'VIC',
ingestion-service |   hazard_status: 'active',
ingestion-service |   alert_level: 'critical',
ingestion-service |   source: 'TEAVS'
ingestion-service | }
ingestion-service | [2026-09-26T23:32:15.364Z] [error] [microservices-backend]: Training model not found
ingestion-service | Received message from core_model_integration_queue: {"url":"https://example.com/e2e-evidence","text":"TEAVS E2E EVIDENCE TEST","timestamp":"2026-09-27T09:50:00+10:00","hazard_type":"phishing","hazard_sever
ity":4,"hazard_timestamp":"2026-09-27T09:49:00+10:00","hazard_location":"VIC","hazard_status":"active","alert_level":"critical","source":"TEAVS"}
ingestion-service | Received core model integration data: {
ingestion-service |   url: 'https://example.com/e2e-evidence',
ingestion-service |   text: 'TEAVS E2E EVIDENCE TEST',
ingestion-service |   timestamp: '2026-09-27T09:50:00+10:00',
ingestion-service |   hazard_type: 'phishing',
ingestion-service |   hazard_severity: 4,
ingestion-service |   hazard_timestamp: '2026-09-27T09:49:00+10:00',
ingestion-service |   hazard_location: 'VIC',
ingestion-service |   hazard_status: 'active',
ingestion-service |   alert_level: 'critical',
ingestion-service |   source: 'TEAVS'
ingestion-service | }
ingestion-service | [2026-09-26T23:37:31.987Z] [error] [microservices-backend]: Training model not found
(base) abdulrehman@Abduls-Air backend %
```

EVID-E2E-04 – Missing model dependency

```

● (base) abdulrehman@Abduls-Air backend % docker compose logs --tail=120 data-ingestion-service | grep -i 'Training model not found'
ingestion-service | [2026-09-26T23:32:15.364Z] [error] [microservices-backend]: Training model not found
ingestion-service | [2026-09-26T23:37:31.987Z] [error] [microservices-backend]: Training model not found
○ (base) abdulrehman@Abduls-Air backend % █

```

EVID-E2E-05 – Model asset search

```

● (base) abdulrehman@Abduls-Air backend % grep -Rni "final_core_xgb_xgboost_trey_xgb_core_v2_epoch_100.joblib" . --exclude-dir=node_modules
./libs/common/src/helper/core_model_integration.py:55:DEFAULT_MODEL_PATH = "final_core_xgb_xgboost_trey_xgb_core_v2_epoch_100.joblib"
./data-ingestion-service/src/services/ingestion.service.ts:188:         "final_core_xgb_xgboost_trey_xgb_core_v2_epoch_100.joblib",
● (base) abdulrehman@Abduls-Air backend % find . -type f \( -name "*.joblib" -o -name "*.pkl" -o -name "*.pickle" \) -print
○ (base) abdulrehman@Abduls-Air backend % █

```

Conclusion

The end-to-end verification demonstrated that the PHOENIX backend can securely accept a TEAVS payload and transport it through the integration pipeline to the core model service.

The integration reached the inference of dependency successfully.

Full model inference could not be completed because the required trained model artifact was not present in the local development environment.

The next step for complete end-to-end verification is to obtain the approved trained model file, load it into the expected storage location, and rerun the same test.